

Universidad Tecnológica de Pereira
Facultad de Ingenierías
Especialización en Tecnologías de la Información y las Comunicaciones
Ciberseguridad

**Actividad #2 – OWASP Juice Shop:
Explotación de Vulnerabilidades Web en Entornos
Controlados**

Daniel Toro Soto

Pereira, Colombia
Mayo de 2026

Índice

1. Introducción	2
2. Configuración del Entorno	2
2.1. Descripción del entorno	2
2.2. Herramientas del entorno	3
3. Tarea 1: SQL Injection — Bypass de Autenticación	5
3.1. Descripción de la vulnerabilidad	5
3.2. Herramientas utilizadas	6
3.3. Pasos realizados	6
3.3.1. Acceso al formulario de login	6
3.3.2. Configuración de Burp Suite como proxy HTTP	7
3.3.3. Inyección del payload	8
3.3.4. Intercepción de la petición POST en Burp Suite	9
3.3.5. Verificación del acceso como administrador	10
3.4. Funcionamiento del ataque	11
3.5. Resultado	12
4. Tarea 2: Sensitive Data Exposure — Directorio /ftp Expuesto	12
5. Tarea 3: Broken Access Control — API de Usuarios sin Autenticación	13

Introducción

La seguridad de las aplicaciones web es uno de los pilares fundamentales de la ciberseguridad moderna. Las organizaciones que desarrollan o mantienen aplicaciones accesibles a través de internet están expuestas a una amplia variedad de ataques que buscan comprometer datos, sesiones de usuario y la integridad de los sistemas subyacentes. En este contexto, el proyecto OWASP (*Open Web Application Security Project*) publica periódicamente el *OWASP Top 10*, una clasificación de las vulnerabilidades de seguridad web más críticas y frecuentes.

En el marco de esta actividad académica, se exploran tres categorías del OWASP Top 10 (2021) mediante la explotación controlada de OWASP Juice Shop, una aplicación web deliberadamente vulnerable diseñada para el entrenamiento en seguridad ofensiva. Las vulnerabilidades abordadas corresponden a las categorías A03 (Inyección), A02 (Fallos Criptográficos / Exposición de Datos Sensibles) y A01 (Control de Acceso Roto).

Todas las pruebas se realizaron en un entorno aislado de red (modo Solo Anfitrión) sobre una Mac Mini M4, utilizando UTM como hipervisor para gestionar las máquinas virtuales Kali Linux (atacante) y Ubuntu 22.04 con Juice Shop (víctima).

Configuración del Entorno

Descripción del entorno

El entorno de laboratorio está compuesto por dos máquinas virtuales ejecutadas en UTM sobre una Mac Mini con procesador Apple M4. Ambas máquinas fueron configuradas con una única interfaz de red en modo Solo Anfitrión (*Host-Only*), lo que garantiza el aislamiento total del tráfico respecto a redes externas.

Cuadro 1: *Configuración de las máquinas virtuales*

Máquina	Rol	SO	Dirección IP
Kali Linux	Atacante	Kali Linux 2024	192.168.40.3
Ubuntu + Juice Shop	Víctima	Ubuntu 22.04	192.168.40.11:3000

Herramientas del entorno

- **UTM:** Hipervisor para macOS/Apple Silicon que permite ejecutar máquinas virtuales x86_64 mediante emulación QEMU.
- **Kali Linux:** Distribución GNU/Linux orientada a pruebas de penetración, que incluye de forma nativa herramientas como Firefox, Burp Suite, curl y sqlmap.
- **OWASP Juice Shop:** Aplicación web Node.js deliberadamente vulnerable, diseñada por OWASP para el entrenamiento en seguridad ofensiva. Corre sobre Express y utiliza una base de datos SQLite.

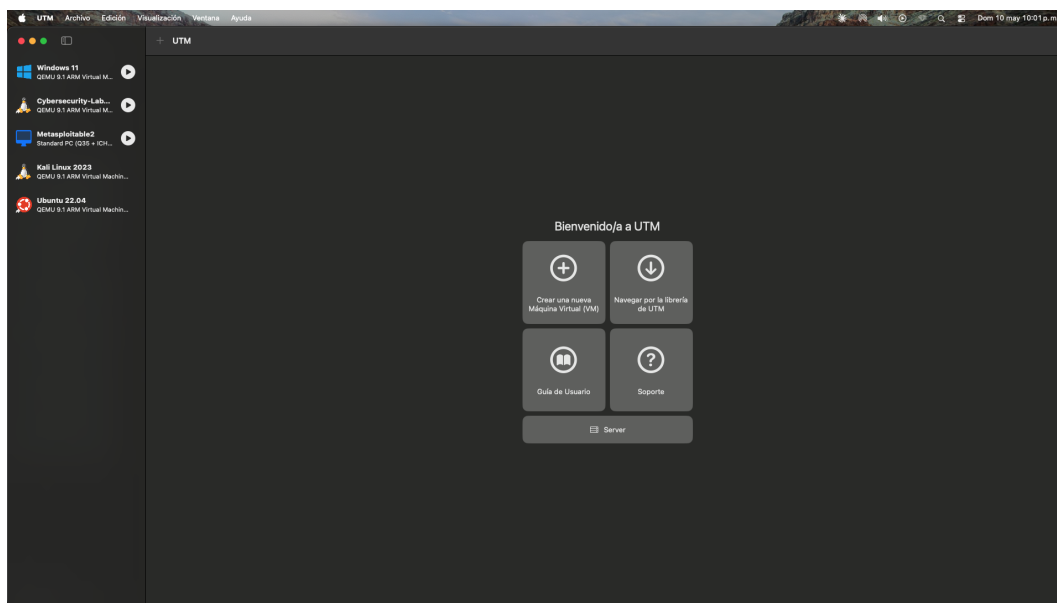
Figura 1: *Vista del hipervisor UTM con las máquinas virtuales activas.*



Figura 2: Máquina virtual Kali Linux (atacante) en ejecución.

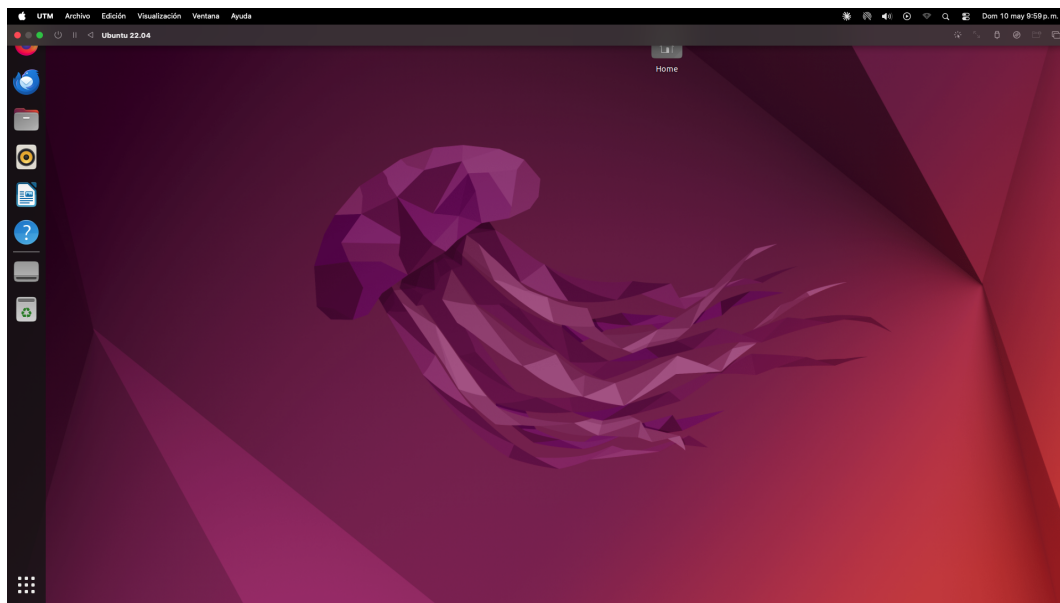


Figura 3: Máquina virtual Ubuntu 22.04 con Juice Shop (víctima) en ejecución.

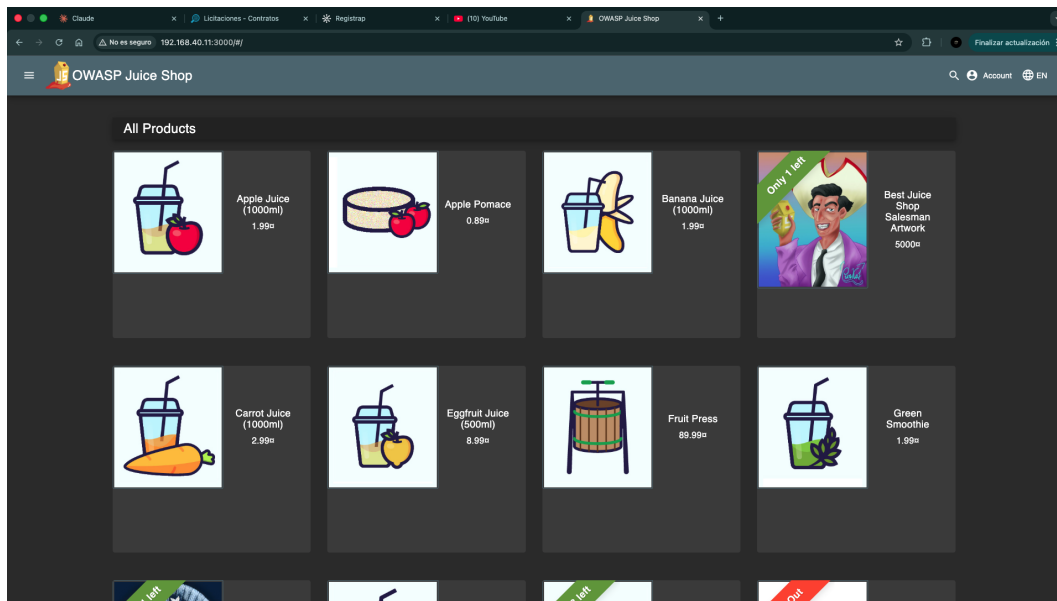


Figura 4: OWASP Juice Shop corriendo en el puerto 3000 de la máquina víctima.

Tarea 1: SQL Injection — Bypass de Autenticación

Descripción de la vulnerabilidad

La inyección SQL (*SQL Injection*) es una vulnerabilidad crítica clasificada bajo **CWE-89** y la categoría **A03 – Inyección** del OWASP Top 10 (2021). Se produce cuando una aplicación web incorpora directamente la entrada del usuario en una consulta SQL sin aplicar sanitización ni usar consultas parametrizadas (*prepared statements*), permitiendo al atacante manipular la lógica de la consulta de base de datos.

En el caso particular del formulario de login de Juice Shop, la consulta SQL que valida las credenciales se construye mediante concatenación directa de los valores ingresados:

```
SELECT * FROM Users WHERE email = '<INPUT_EMAIL>' AND password = '<HASH>'
```

Listing 1: Consulta SQL vulnerable (simplificada)

Al inyectar el payload ' OR 1=1– como valor del campo email, la condición WHERE se transforma en una tautología (siempre verdadera), y los caracteres – comentan el resto de la consulta, eliminando la verificación de contraseña:

```
SELECT * FROM Users WHERE email = '' OR 1=1--' AND password = '<HASH>'
```

Listing 2: Consulta SQL resultante tras la inyección del payload

La base de datos retorna el primer registro de la tabla `Users`, que corresponde al administrador (`admin@juice-sh.op`), autenticando al atacante con privilegios máximos sin conocer ninguna contraseña válida.

Herramientas utilizadas

- **Firefox (Kali Linux):** Navegador web utilizado para interactuar con el formulario de login de Juice Shop e inyectar el payload directamente en el campo de email.
- **Burp Suite Community Edition:** Proxy HTTP utilizado para interceptar y analizar la petición POST generada al enviar el formulario, permitiendo visualizar el payload en tránsito y confirmar la estructura de la solicitud.

Pasos realizados

Acceso al formulario de login

Desde el navegador Firefox en la máquina Kali Linux, se navegó a la página de autenticación de Juice Shop:

```
http://192.168.40.11:3000/#/login
```

Listing 3: URL del formulario de login de Juice Shop

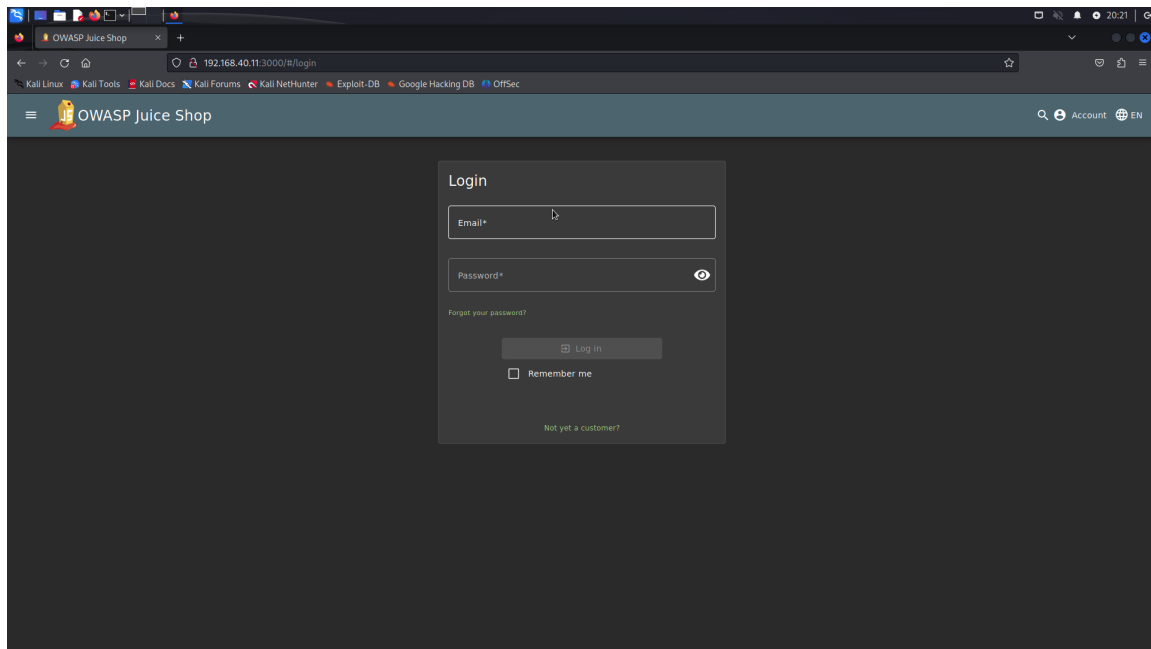


Figura 5: *Página de login de OWASP Juice Shop accedida desde Firefox en Kali Linux.*

Configuración de Burp Suite como proxy HTTP

Se configuró Burp Suite Community Edition como proxy HTTP local en el puerto 8080. En Firefox se habilitó la configuración de proxy manual apuntando a `127.0.0.1:8080`, permitiendo que Burp Suite intercepte todo el tráfico HTTP generado por el navegador. Se activó la opción **Intercept ON** en la pestaña Proxy de Burp Suite.

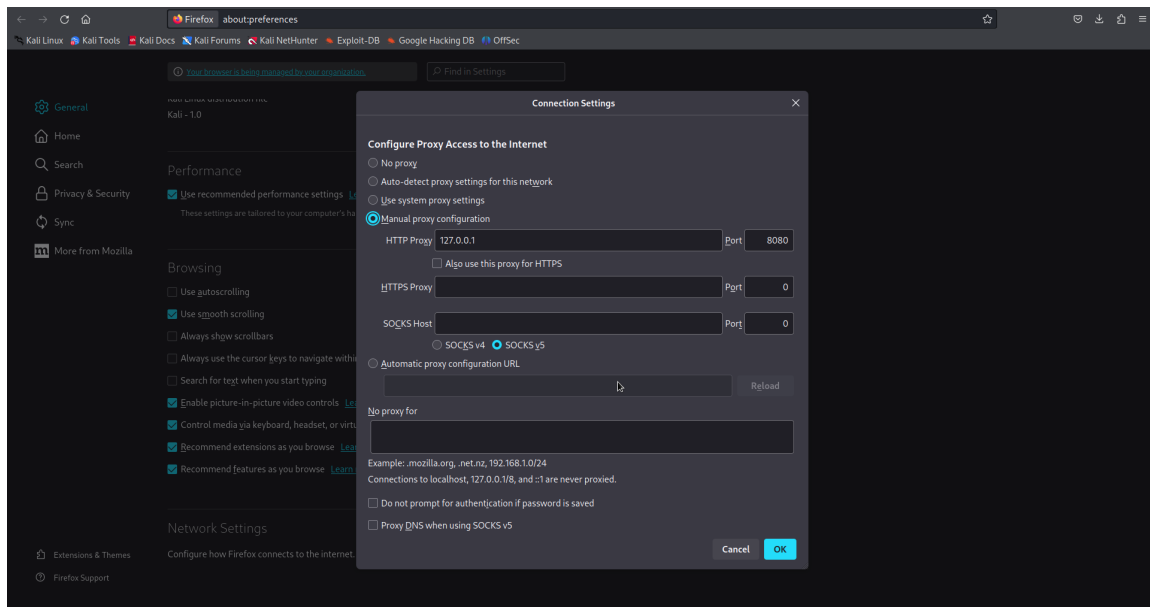


Figura 6: Configuración del proxy manual en Firefox apuntando a Burp Suite en 127.0.0.1:8080.

Inyección del payload

En el campo **Email** del formulario se ingresó el siguiente payload:

```
' OR 1=1--
```

Listing 4: Payload de SQL Injection para bypass de autenticación

En el campo **Password** se ingresó el valor arbitrario `a`. La figura siguiente muestra el formulario con el payload listo para enviar.

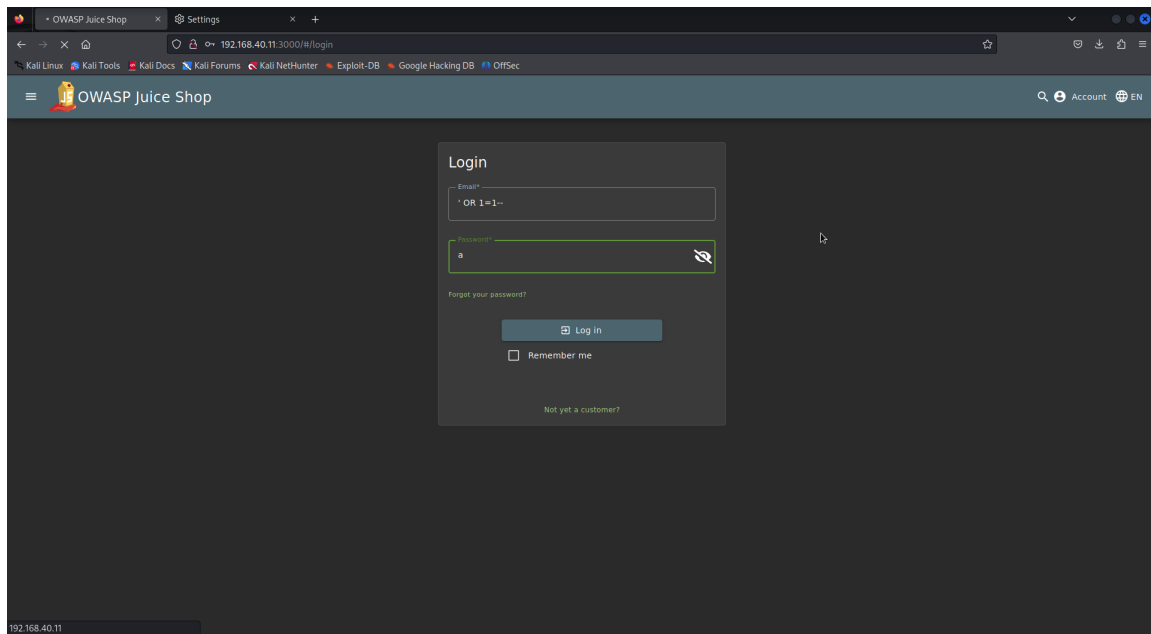
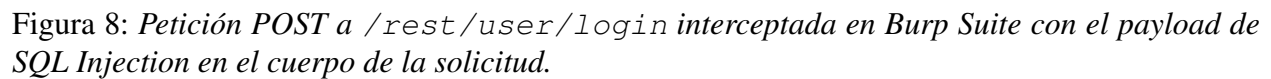


Figura 7: *Formulario de login de Juice Shop con el payload ' OR 1=1-- ingresado en el campo Email y valor arbitrario en Password.*

Intercepción de la petición POST en Burp Suite

Al hacer clic en **Log in**, Burp Suite interceptó la petición POST `/rest/user/login` antes de que llegara al servidor. En la vista del *HTTP history* se observa la petición con el método POST, confirmando que el payload fue enviado en el cuerpo de la solicitud hacia el endpoint de autenticación de la API REST de Juice Shop.



Tras reenviar la petición (*Forward*), Juice Shop procesó la consulta SQL manipulada y autenticó al usuario. La aplicación mostró el banner verde de confirmación del desafío: “*You successfully solved a challenge: Login Admin (Log in with the administrator’s user account.)*”, evidenciando el éxito del ataque.

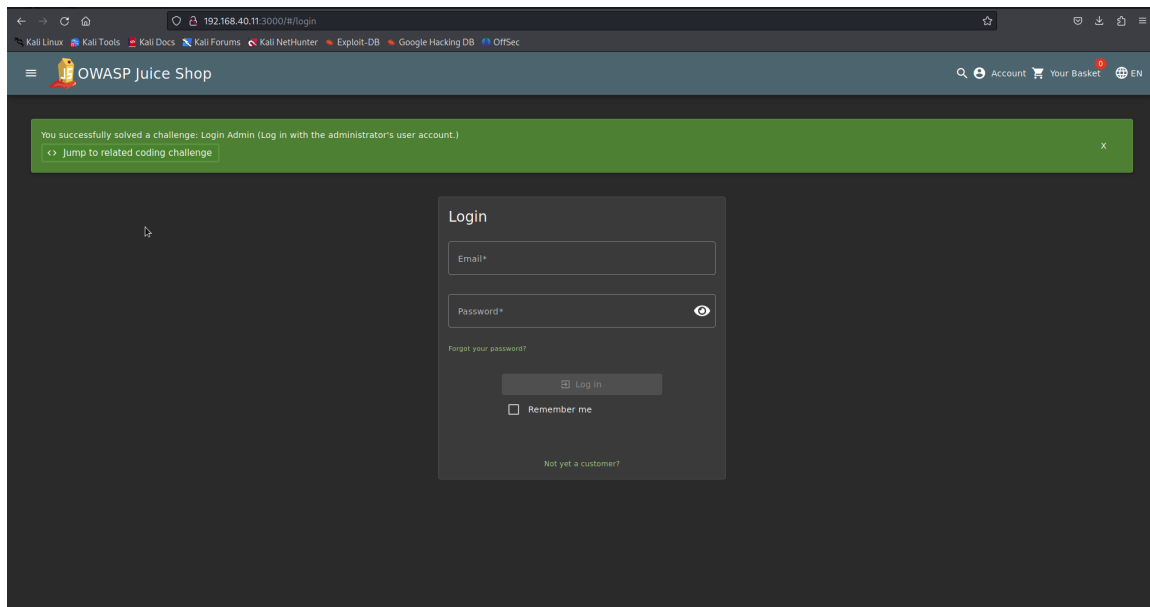


Figura 9: Banner de confirmación de Juice Shop indicando que el desafío “Login Admin” fue resuelto exitosamente mediante SQL Injection.

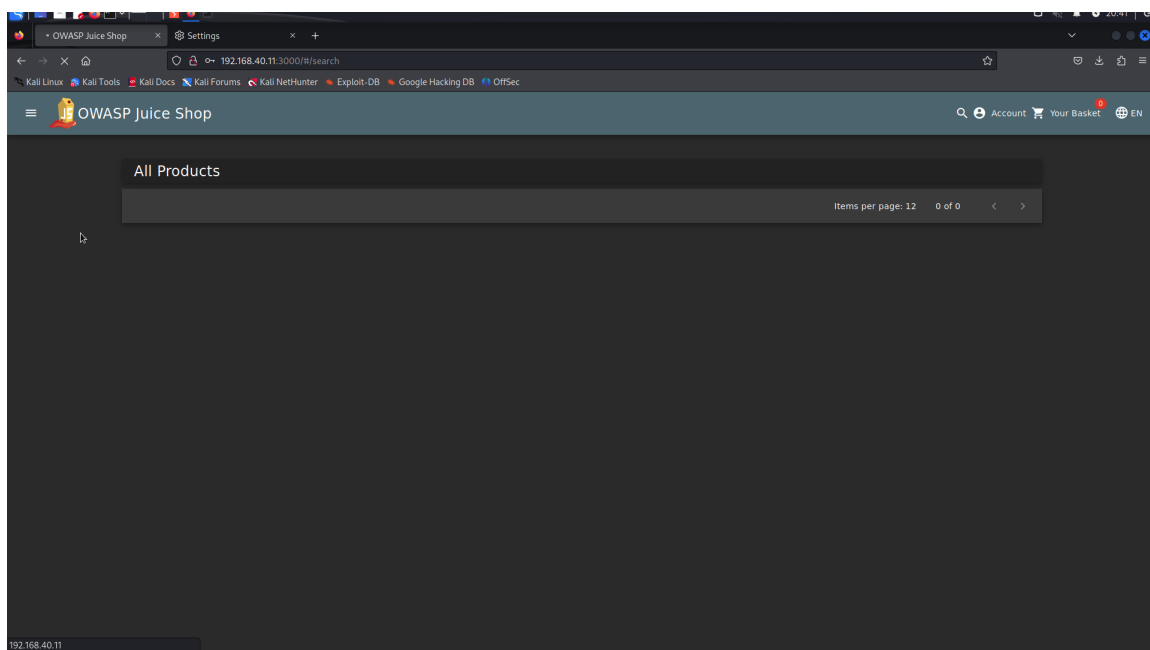


Figura 10: Pantalla principal de Juice Shop tras el bypass exitoso, con sesión activa como administrador.

Funcionamiento del ataque

El ataque de SQL Injection para bypass de autenticación sigue el siguiente flujo:

1. **Identificación del punto de inyección:** El formulario de login acepta los campos `email` y `password`, que son incorporados sin sanitización en la consulta SQL de verificación de credenciales.
2. **Construcción del payload:** El atacante ingresa `' OR 1=1-` como valor del email. La comilla simple (`'`) cierra prematuramente el literal de cadena iniciado por la aplicación. La expresión `OR 1=1` convierte la condición `WHERE` en una tautología. Los guiones dobles (`-`) inician un comentario SQL que descarta el resto de la consulta original, incluyendo la verificación de contraseña.
3. **Ejecución de la consulta manipulada:** La base de datos SQLite evalúa la condición `WHERE email = " OR 1=1`, que es siempre verdadera para cualquier fila. Retorna el primer usuario registrado en la tabla, que es el administrador.
4. **Autenticación como administrador:** La aplicación, al recibir un registro de usuario válido, establece una sesión autenticada con los privilegios del usuario retornado (`admin@juice-sh.op`).

Resultado

Se logró el inicio de sesión exitoso como administrador (`admin@juice-sh.op`) en OWASP Juice Shop sin conocer ni proporcionar ninguna contraseña válida. El ataque explotó la ausencia de consultas parametrizadas en el módulo de autenticación, demostrando el impacto crítico de la vulnerabilidad CWE-89 (A03 – Inyección). Un atacante real podría aprovechar este acceso para gestionar usuarios, acceder a órdenes de compra, modificar precios, y obtener información confidencial de todos los clientes registrados en la plataforma.

Tarea 2: Sensitive Data Exposure — Directorio /ftp Expuesto

(Pendiente de documentación)

Tarea 3: Broken Access Control — API de Usuarios sin Autenticación

(Pendiente de documentación)